

Choices for `make_optional` and `value_or()`

Document #:

P3199R0

Date:

2024-03-22

Audience:

LEWG

Reply-to:

[Steve Downey <sdowney@gmail.com>](mailto:sdowney@gmail.com)

Source:

<https://github.com/steve-downey/wg21org>

homework-tokyo.org

tags/P3199R0-0-g6bc237b

Table of Contents

1 From the design notes slides

1.1 `make_optional`

1.2 `value_or`

2 `std::make_optional` in the context of `optional<T&>`

2.1 Recommend **Do Nothing**

3 `std::optional<T&>::value_or`

3.1 Standard for `optional<T>::value_or`

3.2 Boost

3.3 TI-optional

3.4 Flux

3.5 Think-Cell

3.6 Summary

3.7 Proposal

3.7.1 Examples

3.7.2 Motivation for reference returning `value_or`

3.7.3 Discussion

3.7.4 Proposal

4 References

Abstract: Homework: Review options for `std::make_optional` and `std::optional<T&>::value_or()`

1. From the design notes slides

Recap of 2024-03-20 presentation slides that did not have consensus.

1.1. `make_optional`

- Because of existing code, `make_optional<T&>` must return `optional<T>` rather than `optional<T&>`.
- Returning `optional<T&>` is consistent and defensible, and a few optional implementations in production make this choice.
- It is, however, quite easy to construct a `make_optional` expression that deduces a different category causing possibly dangerous changes to code.

1.2. `value_or`

Have `value_or` return a `T&`.

A reference so that there is a shared referent for the or side as well as the optional.

Check that the supplied value can be bound to a `T&`.

2. `std::make_optional` in the context of `optional<T>`

Current specification:

```
template<class T> constexpr optional<decay_t<T>> make_optional(T&& v);  
// Returns: optional<decay_t<T>>(std::forward<T>(v)).  
  
template<class T, class...Args>  
    constexpr optional<T> make_optional(Args&&... args);  
// Effects: Equivalent to: return optional<T>(in_place, std::forward<A  
  
template<class T, class U, class... Args>  
    constexpr optional<T> make_optional(initializer_list<U> il, Args&&..  
// Effects: Equivalent to: return optional<T>(in_place, il, std::forwa
```

The second two forms are disallowed for `std::optional<T&>` because `in_place` with multiple arguments does not make sense.

```
#include <optional>

struct MyStruct {};

MyStruct& func();

static_assert(std::is_same_v<
               std::optional<MyStruct>,
               decltype(std::make_optional(
                           MyStruct{}))> == true);

static_assert(std::is_same_v<
               std::optional<MyStruct>,
               decltype(std::make_optional(
                           func()))> == true);

std::optional<MyStruct> r1 =
    std::make_optional<MyStruct>(func());

std::optional<MyStruct> r1a =
    std::make_optional<MyStruct&>(func());

std::optional<MyStruct> r1b =
    std::make_optional(func());

std::optional<MyStruct> r2 =
    std::make_optional<MyStruct>(
        std::move(MyStruct{}));

std::optional<MyStruct> r2a =
    std::make_optional<MyStruct&&>(
        std::move(MyStruct{}));
```

<https://compiler-explorer.com/z/no7znrz9v>

We can currently spell as well as deduce `MyStruct&` and `MyStruct&&` for `make_optional` and the result is a `std::optional<T>`.

I think it is clear that changing the return type for a `make_optional` on an expression that has a reference type from a `std::optional<T>` to a `std::optional<T&>` would not be acceptable in existing code. The code might fail to compile because we forbid dangling temporary binding, but successful compilation might be even worse.

It's not clear right now that people spelling the template with a `T&` are doing so with great deliberateness. There is certainly the possibility that users may be confused and try to get an `optional<T&>` out of `make_optional`, but failure should be generally quickly visible as there is no viable path to construct or assign a `optional<T>` to an `optional<T&>`. Conversions do permit an `optional<T>` to be initialized from an `optional<T&>`, as `optional` is fairly permissive about conversions, and this seems mostly desirable.

```
std::optional<int> x;  
std::optional<int&> x2 = x;    //fails  
std::optional<int&> x3{x};    //fails  
i3 = x;                      //fails  
  
std::optional<int&> k;  
std::optional<int&> y = k;    // compiles  
std::optional<int&> y2{k};    // compiles  
y = k;                      // compiles
```

Given the adopted policy regarding `[[nodiscard]]` I am not sure we should mandate a diagnostic at this point, without at least more feedback from standard library implementors. Policy paper to come.

2.1. Recommend **Do Nothing**

Make no changes to the behavior or compilation of `std::make_optional`. It's not clear right now we need a `make_optional_ref` in place of the existing constructors. There's no constructor confusion, or multi-arg emplace. I think I would need evidence that `std::optional<MyType&>{}` is not sufficient.

3. `std::optional<T>::value_or`

There are different implementations in the wild that both support references and support `value_or`.

3.1. Standard for `optional<T>::value_or`

(Köppe 2022)

```
template<class U> constexpr T value_or(U&& v) const &;

// Mandates: is_copy_constructible_v<T> && is_convertible_v<U&&, T> is
// Effects: Equivalent to:
//   return has_value() ? *this : static_cast<T>(std::forward<U>(v));

template<class U> constexpr T value_or(U&& v) &&;

// Mandates: is_move_constructible_v<T> && is_convertible_v<U&&, T> is
// Effects: Equivalent to:
//   return has_value() ? std::move(*this) : static_cast<T>(std::forwa
```

Note that for `optional<T>` moving the value out of a held value in an rvalue-ref optional is entirely reasonable.

It is not for a reference semantic optional.

3.2. Boost

([Boost, n.d.](#))

```
template<class U> T optional<T>::value_or(U && v) const& ;

// Effects: Equivalent to if (*this) return **this; else return std::f

// Remarks: If T is not CopyConstructible or U && is not convertible t
// program is ill-formed. Notes: On compilers that do not support
// ref-qualifiers on member functions this overload is replaced with t
// const-qualified member function. On compilers without rvalue refere
// support the type of v becomes U const&.

template<class U> T optional<T>::value_or(U && v) && ;

// Effects: Equivalent to if (*this) return std::move(**this); else re

// Remarks: If T is not MoveConstructible or U && is not convertible t
// program is ill-formed. Notes: On compilers that do not support
// ref-qualifiers on member functions this overload is not present.

template<class R> T& optional<T&>::value_or( R&& r ) const noexcept;

// Effects: Equivalent to if (*this) return **this; else return r;.
// Remarks: Unless R is an lvalue reference, the program is ill-formed
```

3.3. TI-optional

([Brand, n.d.](#))

```
template <class U> constexpr T optional<T>::value_or(U &&u) && noexc
```

Returns a `T` rather than a `T&`

3.4. Flux

(Brindle, n.d.) This is from Tristan Brindle's tristanbrindle.com/flux/

```
#define FLUX_FWD(x) static_cast<decltype(x)&&>(x)
//...
// optional<T&>
[[nodiscard]]
constexpr auto value_unchecked() const noexcept -> T& { return *pt

[[nodiscard]]
constexpr auto value_or(auto&& alt) const
    -> decltype(has_value() ? value_unchecked() : FLUX_FWD(alt))
{
    return has_value() ? value_unchecked() : FLUX_FWD(alt);
}
```

Flux returns references, but effectively returns a common reference type.

Note that all implementations return a `T&` from `value()`, as well as for `operator*()` for all template instantiations. Arguing that `value_or` should return `T` because ``value`` is plausible, but not supportable for existing APIs.

3.5. Think-Cell

(GmbH, n.d.)

<https://github.com/think-cell/think-cell-library/>

`value_or` of both `optional<T>` and `optional<T&>` returns `tc::common_reference<decltype(value()), U&&>`, which is like `std::common_reference`, but doesn't compile for e.g. long and unsigned long).

see:

- [tc optional value_or](#)
- [Macro definition used](#)
- [tc common reference](#)

3.6. Summary

Impl	Behavior
Standard	optional<T>::value_or returns a T
Boost	optional<T>::value_or returns a T
	optional<T&>::value_or returns a T&
TL	optional<T>::value_or returns a T
	optional<T&>::value_or returns a T
Flux	returns result of ternary, similar to common_reference
Think-cell	returns common_reference, with some caveats

3.7. Proposal

Last night on Mattermost Tomasz Kamiński proposed

```
template <class U, class R = std::common_reference_t<T&, U&&>>
auto value_or(U&& v) const -> R {
    static_assert(!std::reference_constructs_from_temporary_v<R, U
    static_assert(!std::reference_constructs_from_temporary_v<R, T
    return ptr ? static_cast<R>(*ptr) : static_cast<R>((U&&)v);
}
```

3.7.1. Examples

```
optional<int&> o; // disengaged optional<int&>
long i{42};
auto&& val = o.value_or(i);
static_assert(std::same_as<decltype(o.value()), int&>);
static_assert(std::same_as<decltype(o.value_or(i)), long>);

optional<base&> b;
derived d;
static_assert(std::same_as<decltype(b.value()), base&>);
static_assert(std::same_as<decltype(b.value_or(d)), base&>);
```

<https://godbolt.org/z/rWo7Wvd6b>

3.7.2. Motivation for reference returning value_or

```
struct Logger {
    virtual void debug(std::string_view sv) = 0;
};

struct DefaultLogger : public Logger {
    DefaultLogger() {}
    DefaultLogger(const DefaultLogger & l) = delete;
    virtual void debug(std::string_view sv) override {}
};

DefaultLogger& getDefaultLogger() {
    static DefaultLogger dl;
    return dl;
}

Logger& getLogger(optional<Logger&> logger) {
    return l.value_or(getDefaultLogger());
}
```

3.7.3. Discussion

I believe that `std::optional<T>::value_or` returning a `T` is an unfortunate and unfixable mistake. Others believe that instead there ought to have been a `value()` returning `T`, and a `ref()` returning `T&`. The ship for changing those has long since sailed.

I believe the use case of alternative references is important, and should be supported. I have been convinced that `value_or` is not an available name for that function.

However, given the state of `std::optional<T>::value_or`, I think this function needs to be called `ref_or`.

3.7.4. Proposal

We should instead remove `value_or`. There is no clear correct answer that works generically. Conversions from `std::optional<reference_wrapper<T>>` already need to do some work, as do conversions from any other existing optional. Making that work clear is a benefit.

As a fallback, have `value_or` return a prvalue, a `T`. A `T&`, instead of `std::common_reference_t<T&, U>`, excludes too many reasonable cases.

4. References

Boost. n.d. “Detailed Semantics - Optional Values - 1.82.0.”

https://www.boost.org/doc/libs/1_82_0/libs/optional/doc/html/boost_optional/reference/header___boost_optional_optional_hpp_/detailed_semantics___optional_values.html#reference_optional_value_or.

Brand, Sy. n.d. “TartanLlama/Optional: C++11/14/17 Std:Optional with Functional-Style Extensions and Reference Support.”

<https://github.com/TartanLlama/optional>.

Brindle, Tristan. n.d. “Tcbrindle/Flux: A C++20 Library for Sequence-Oriented Programming.” <https://github.com/tcbrindle/flux>.

Downey, Steve, and Peter Sommerlad. 2024. “P2988R1: Std:Optional.” <https://wg21.link/p2988r1>; WG21.

GmbH, think-cell Software. n.d. "Think-Cell/Think-Cell-Library: Think-Cell Core Library." <https://github.com/think-cell/think-cell-library/>.

Köppe, Thomas. 2022. "N4928: Working Draft, Standard for Programming Language c++." <https://wg21.link/n4928>; WG21.

Exported: 2024-03-22 16:22:48